DESIGN AND ANALYSIS OF ALGORITHMS · UNIT 2

Quick Sort

Divide and conquer, where the partition does the work

Ammey Kuraaday
Roll No. B107

Shreyash Javale
Roll No. B34

After one partition, the pivot is home



sorted on its own | **final position** | sorted on its own

	Merge sort	Quick sort
Divide	Trivial: cut the array in the middle	**The real work: partition around a pivot**
Combine	**The real work: merge two sorted halves**	Nothing. The array is already in order

Quick sort moves the effort from combine to divide.

Partition: two pointers, one swap at the end

```
1  Partition(A, low, high)
2    pivot = A[low]
3    i = low + 1
4    j = high
5    while true
6      while A[i] <= pivot: i++
7      while A[j] >  pivot: j--
8      if i < j: swap(A[i], A[j])
9      else: break
10   swap(A[low], A[j])
11   return j
```

i Scans right. Stops at the first element greater than the pivot.

j Scans left. Stops at the first element not greater than the pivot.

swap If i < j, swap A[i] and A[j], then keep scanning.

end Once they cross, swap the pivot into A[j]. That index is final.

Bounds bug in the notes. If the pivot is the largest value, i runs past the end: on [85 80 75 70] (indices 5 to 8) it reads A[9]. Fix: **`while i <= high and A[i] <= pivot`**

Partition on [65, 70, 75, 80, 85, 60, 55, 50, 45]

	0	1	2	3	4	5	6	7	8	
start	**65**	70	75	80	85	60	55	50	45	Pivot = A[0] = 65
swap 1	**65**	**45**	75	80	85	60	55	50	**70**	i stops at 70, j stops at 45: swap
swap 2	**65**	45	**50**	80	85	60	55	**75**	70	i stops at 75, j stops at 50: swap
swap 3	**65**	45	50	**55**	85	60	**80**	75	70	i stops at 80, j stops at 55: swap
swap 4	**65**	45	50	55	**60**	**85**	80	75	70	i stops at 85, j stops at 60: swap
finish	60	45	50	55	**65**	85	80	75	70	**i = 5 has passed j = 4: swap the pivot with A[4]**

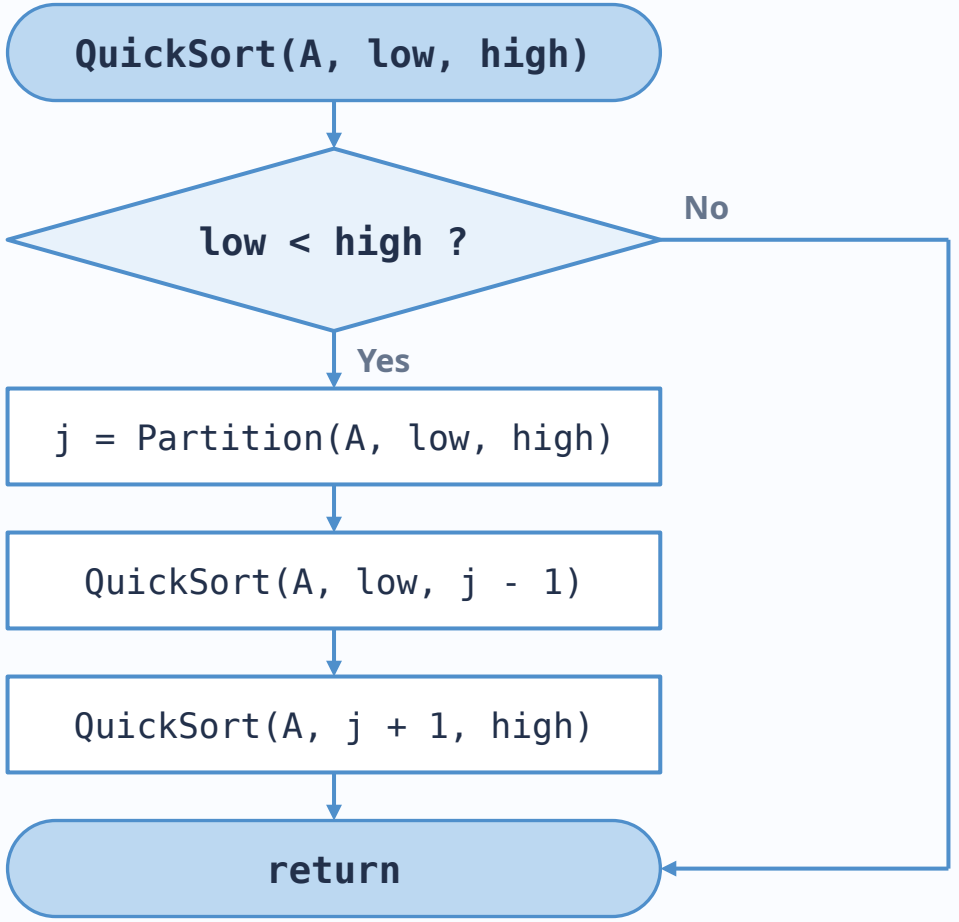
pivot · swapped pair · final row: left four ≤ 65, right four ≥ 65

QuickSort: partition, then recurse twice

```
1  QuickSort(A, low, high)
2    if low < high
3      j = Partition(A, low, high)
4      QuickSort(A, low, j - 1)
5      QuickSort(A, j + 1, high)
```

No combine step

When both recursive calls return, the whole range is already sorted. This is the key difference from merge sort.



Running time depends on the pivot

BEST CASE

$T(n) = 2T(n/2) + n$

$\Theta(n \log n)$

The pivot lands in the middle every time: log n levels with n work on each.

AVERAGE CASE

random pivot

$\approx 1.39\ n \log n$

Only about 39% above the minimum any comparison sort needs.

WORST CASE

$T(n) = T(n - 1) + n$

$\Theta(n^2)$

The pivot is always the smallest or largest value. Sorted input with a first-element pivot does this.

Fixing the worst case

Use a random pivot, or the median of three (≈ 1.19 n log n). On 5,000 sorted values a first-element pivot needs 12,497,500 comparisons, against about 61,000 for n log n.

Why quick sort wins in practice

Introsort

`C++ std::sort`

Quick sort that falls back to heap sort if recursion gets too deep, and uses insertion sort on tiny ranges.

pdqsort

`Go sort, Rust sort_unstable`

Spots bad pivots and ordered patterns, then changes strategy, so the worst case is rare.

Dual-pivot

`Java Arrays.sort`

Two pivots give three parts. Java has used it for sorting primitive arrays since version 7.

Takeaways

Partition does the work, and there is no combine step.
Best and average case n log n. Worst case n^2, on sorted input with a first-element pivot.
In place and cache friendly, with the worst case engineered away.

Thank you · Questions?

Ammey Kuraaday (B107) · Shreyash Javale (B34)